

Projet C - Séquence 1.3 - Script SQL de la base

Voici le script SQL pour Supabase correspondant à votre **Modèle Conceptuel de Données (MCD)**. Ce script crée les tables **Utilisateur**, **Sujet** et **Commentaire**, ainsi que les relations et contraintes associées.

Je vais générer un canvas avec le script SQL complet et optimisé pour Supabase, incluant :

- Les tables avec leurs colonnes et types de données.
- Les clés primaires et étrangères.
- Les contraintes **UNIQUE** et **NOT NULL**.
- Les relations 1-N entre les entités.

```
-- Extension UUID si non activée par défaut
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

-- Table Utilisateur
CREATE TABLE Utilisateur (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email VARCHAR(255) UNIQUE NOT NULL,
  mot_de_passe VARCHAR(255) NOT NULL,
  date_inscription TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

-- Table Sujet
CREATE TABLE Sujet (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  titre VARCHAR(255) NOT NULL,
  contenu TEXT NOT NULL,
  date_creation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  auteur_id UUID NOT NULL REFERENCES Utilisateur(id) ON DELETE CASCADE
);

-- Table Commentaire
CREATE TABLE Commentaire (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  contenu TEXT NOT NULL,
  date_creation TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  auteur_id UUID NOT NULL REFERENCES Utilisateur(id) ON DELETE CASCADE,
  sujet_id UUID NOT NULL REFERENCES Sujet(id) ON DELETE CASCADE
);

-- Index pour optimiser les requêtes sur les clés étrangères
CREATE INDEX idx_sujet_auteur_id ON Sujet(auteur_id);
CREATE INDEX idx_commentaire_auteur_id ON Commentaire(auteur_id);
CREATE INDEX idx_commentaire_sujet_id ON Commentaire(sujet_id);
```

```

-- Trigger pour mettre à jour la date de création automatiquement
CREATE OR REPLACE FUNCTION update_timestamp()
RETURNS TRIGGER AS $$
BEGIN
    NEW.date_creation = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Appliquer le trigger aux tables Sujet et Commentaire
DROP TRIGGER IF EXISTS trigger_update_sujet_timestamp ON Sujet;
CREATE TRIGGER trigger_update_sujet_timestamp
    BEFORE INSERT OR UPDATE ON Sujet
    FOR EACH ROW
    EXECUTE FUNCTION update_timestamp();

DROP TRIGGER IF EXISTS trigger_update_commentaire_timestamp ON Commentaire;
CREATE TRIGGER trigger_update_commentaire_timestamp
    BEFORE INSERT OR UPDATE ON Commentaire
    FOR EACH ROW
    EXECUTE FUNCTION update_timestamp();

-- Activation du RLS pour toutes les tables
ALTER TABLE Utilisateur ENABLE ROW LEVEL SECURITY;
ALTER TABLE Sujet ENABLE ROW LEVEL SECURITY;
ALTER TABLE Commentaire ENABLE ROW LEVEL SECURITY;

-- =====
-- Règles pour la table Utilisateur
-- =====

-- Lecture : Un utilisateur ne peut voir que son propre profil
CREATE POLICY "Utilisateur: Lecture propre profil"
ON Utilisateur
FOR SELECT
USING (auth.uid() = id::uuid);

-- Insertion : Autoriser la création de nouveaux utilisateurs (inscription)
CREATE POLICY "Utilisateur: Insertion"
ON Utilisateur
FOR INSERT
WITH CHECK (true);

-- Mise à jour : Un utilisateur ne peut modifier que son propre profil
CREATE POLICY "Utilisateur: Mise à jour propre profil"
ON Utilisateur
FOR UPDATE
USING (auth.uid() = id::uuid);

-- Suppression : Un utilisateur ne peut supprimer que son propre profil
CREATE POLICY "Utilisateur: Suppression propre profil"
ON Utilisateur
FOR DELETE
USING (auth.uid() = id::uuid);

```

```

-- =====
-- Règles pour la table Sujet
-- =====

-- Lecture : Tous les utilisateurs peuvent voir tous les sujets
CREATE POLICY "Sujet: Lecture tous"
ON Sujet
FOR SELECT
USING (true);

-- Insertion : Un utilisateur peut créer un sujet (lié à son auteur_id)
CREATE POLICY "Sujet: Insertion"
ON Sujet
FOR INSERT
WITH CHECK (auth.uid() = auteur_id::uuid);

-- Mise à jour : Un utilisateur ne peut modifier que ses propres sujets
CREATE POLICY "Sujet: Mise à jour propres sujets"
ON Sujet
FOR UPDATE
USING (auth.uid() = auteur_id::uuid);

-- Suppression : Un utilisateur ne peut supprimer que ses propres sujets
CREATE POLICY "Sujet: Suppression propres sujets"
ON Sujet
FOR DELETE
USING (auth.uid() = auteur_id::uuid);

-- =====
-- Règles pour la table Commentaire
-- =====

-- Lecture : Tous les utilisateurs peuvent voir tous les commentaires
CREATE POLICY "Commentaire: Lecture tous"
ON Commentaire
FOR SELECT
USING (true);

-- Insertion : Un utilisateur peut ajouter un commentaire (lié à son auteur_id et à un sujet_id existant)
CREATE POLICY "Commentaire: Insertion"
ON Commentaire
FOR INSERT
WITH CHECK (
    auth.uid() = auteur_id::uuid
    AND EXISTS (SELECT 1 FROM Sujet WHERE id = sujet_id)
);

-- Mise à jour : Un utilisateur ne peut modifier que ses propres commentaires
CREATE POLICY "Commentaire: Mise à jour propres commentaires"
ON Commentaire
FOR UPDATE
USING (auth.uid() = auteur_id::uuid);

-- Suppression : Un utilisateur ne peut supprimer que ses propres commentaires
CREATE POLICY "Commentaire: Suppression propres commentaires"
ON Commentaire

```

```
FOR DELETE
USING (auth.uid() = auteur_id::uuid);
-
```

Le script SQL pour Supabase a été généré dans le canvas ci-dessus. Il inclut :

- **Les tables** : **Utilisateur**, **Sujet**, et **Commentaire** avec leurs colonnes et contraintes.
- **Les relations** : Clés étrangères pour lier les utilisateurs aux sujets et aux commentaires.
- **Les index** : Pour optimiser les requêtes sur les clés étrangères.
- **Les triggers** : Pour mettre à jour automatiquement les timestamps à chaque insertion ou mise à jour.

Résultat RLS

Feedback

Search...

Auto-enable RLS for new tables

Create an event trigger that enables Row Level Security on all new tables

Learn more

schema public

Filter tables and policies

commentaire

Disable RLS

Create policy

NAME	COMMAND	APPLIED TO	
Commentaire: Insertion	INSERT	public	
Commentaire: Lecture tous	SELECT	public	
Commentaire: Mise à jour propres commentaires	UPDATE	public	
Commentaire: Suppression propres commentaires	DELETE	public	

sujet

Disable RLS

Create policy

NAME	COMMAND	APPLIED TO	
Sujet: Insertion	INSERT	public	
Sujet: Lecture tous	SELECT	public	
Sujet: Mise à jour propres sujets	UPDATE	public	
Sujet: Suppression propres sujets	DELETE	public	

utilisateur

Disable RLS

Create policy

NAME	COMMAND	APPLIED TO	
Utilisateur: Insertion	INSERT	public	
Utilisateur: Lecture propre profil	SELECT	public	
Utilisateur: Mise à jour propre profil	UPDATE	public	
Utilisateur: Suppression propre profil	DELETE	public	

PRE

Row

Veri

by r

En

Résultat Schéma Database

